

Python Task 02

Decision Making & Logic Building

Cyber Security Summer Internship

Part A

Even or Odd Checker

`even_odd.py`

This program takes a number from the user and uses the modulo operator to determine whether the number is even or odd.

Code

```
print("=" * 40)
print("      EVEN OR ODD CHECKER")
print("=" * 40)

try:
    number = int(input("Enter a number: "))
    if number % 2 == 0:
        print(f"\n{number} is an Even Number")
    else:
        print(f"\n{number} is an Odd Number")
except ValueError:
    print("Invalid input. Please enter a whole number.")
```

Step-by-Step Explanation

- | | |
|---|---|
| 1 | <code>input()</code> reads whatever the user types and returns it as a string. |
| 2 | <code>int()</code> converts that string into a whole number so arithmetic can be performed on it. |
| 3 | The <code>%</code> (modulo) operator calculates the remainder after dividing by 2. Any even number divided by 2 leaves remainder 0; odd numbers leave remainder 1. |
| 4 | <code>if / else</code> checks the remainder and prints the correct label — Even or Odd. |
| 5 | <code>try / except ValueError</code> catches the crash that would happen if the user typed letters instead of a number, and shows a friendly error message instead. |

Sample Output

```
Enter a number: 15

15 is an Odd Number
```

Key Concepts

Concept	How it is used
int()	Converts a string to an integer so arithmetic works on user input
% modulo operator	Returns the remainder of a division — 0 means even, 1 means odd
if / else	Runs different code based on whether the condition is true or false
try / except	Handles errors gracefully without crashing the program

This program accepts three numbers from the user and finds the largest by comparing them using chained if / elif / else conditions.

Code

```
try:
    a = float(input("Enter First Number : "))
    b = float(input("Enter Second Number : "))
    c = float(input("Enter Third Number : "))

    if a >= b and a >= c:
        largest = a
    elif b >= a and b >= c:
        largest = b
    else:
        largest = c

    fmt = lambda n: int(n) if n == int(n) else n
    print(f"\nLargest Number = {fmt(largest)}")
except ValueError:
    print("Invalid input.")
```

Step-by-Step Explanation

1	float() is used instead of int() so the program also works with decimal numbers like 10.5.
2	The first if checks whether a is greater than or equal to both b and c. The keyword and means both conditions must be true at the same time.
3	elif (else-if) is only checked when the first condition is false. It tests whether b is the largest.
4	The final else means if neither a nor b won, then c must be the largest — no extra comparison needed.
5	The lambda helper removes the unnecessary .0 decimal when the result is a whole number (shows 25 instead of 25.0).

Sample Output

```
Enter First Number : 10
Enter Second Number : 25
Enter Third Number : 18

Largest Number = 25
```

Key Concepts

Concept	How it is used
float()	Converts input to a decimal number, supporting both integers and fractions
and operator if / elif / else	Both conditions must be true simultaneously for the branch to run A chain of checks — Python stops and runs only the first true branch
lambda	A short one-line function used here to clean up the number display

This program takes a marks value between 0 and 100, then maps it to a letter grade and a remark using an elif ladder.

Grading Criteria

Marks	Grade	Remark
90 – 100	A	Excellent!
80 – 89	B	Very Good!
70 – 79	C	Good
60 – 69	D	Needs Improvement
Below 60	F	Failed

Code

```
try:
    marks = float(input("Enter your marks (0 - 100): "))

    if marks < 0 or marks > 100:
        print("Marks must be between 0 and 100.")
    elif marks >= 90:
        grade, remark = "A", "Excellent!"
    elif marks >= 80:
        grade, remark = "B", "Very Good!"
    elif marks >= 70:
        grade, remark = "C", "Good"
    elif marks >= 60:
        grade, remark = "D", "Needs Improvement"
    else:
        grade, remark = "F", "Failed"

    if 0 <= marks <= 100:
        print(f"Grade : {grade} --- {remark}")
except ValueError:
    print("Invalid input.")
```

Step-by-Step Explanation

- 1 The first if is a guard — it immediately rejects marks below 0 or above 100 before any grading runs.
- 2 Python checks each elif from top to bottom and stops at the first true one. So elif marks >= 80 only runs when the mark is already known to be below 90.
- 3 The final else acts as a catch-all — if none of the conditions matched, the mark is below 60, so grade F is assigned automatically.

4

Two variables (grade and remark) are assigned together on one line using tuple unpacking — a clean Python shortcut.

5

try / except catches non-numeric input so the program does not crash if the user types text.

Sample Output

```
Enter your marks (0 - 100): 85
```

```
Marks : 85.0
```

```
Grade : B
```

```
Remark : Very Good!
```

Key Concepts

Concept	How it is used
elif ladder	Checks ranges in order and stops at the first condition that is true
or operator	True if either condition is true — used to catch marks outside 0–100
else (catch-all)	Runs when no previous condition matched
Tuple unpacking	Assigns two variables on one line: grade, remark = "B", "Very Good!"

This program stores a predefined username and password, takes input from the user, and validates the credentials using string comparison.

Code

```
STORED_USERNAME = "admin"
STORED_PASSWORD = "password123"

print("=" * 40)
print("          LOGIN PORTAL")
print("=" * 40)

username = input("Username : ")
password = input("Password : ")

if username == STORED_USERNAME and password == STORED_PASSWORD:
    print("Login Successful!")
    print(f"Welcome, {username}!")
else:
    print("Invalid Credentials")
    print("Please check your username and password.")
```

Step-by-Step Explanation

- | | |
|---|---|
| 1 | The correct username and password are stored at the top in ALL_CAPS variables. In Python, ALL_CAPS signals that a value is a constant that should not be changed. |
| 2 | input() collects the username and password the user types. Both are stored as plain strings. |
| 3 | The == operator compares two strings character by character. Even one wrong character — wrong case, extra space — makes the comparison return False. |
| 4 | The and operator ensures both the username and password must match. If either one is wrong, the login fails. |
| 5 | The else branch handles every other case — wrong username, wrong password, or both wrong. |

Sample Output — Correct Credentials

```
Username : admin
Password : password123
-----
Login Successful!
Welcome, admin!
```

Sample Output — Wrong Credentials

```
Username : admin
Password : wrongpass
-----
Invalid Credentials
Please check your username and password.
```

Key Concepts

Concept	How it is used
Constants (ALL_CAPS)	Convention to signal a variable whose value should not change
== string comparison	Checks if two strings are exactly identical, character by character
and operator	Both username and password must match for login to succeed
String case sensitivity	"Admin" and "admin" are different strings in Python

This program analyses a password against five security criteria and rates it as Weak, Moderate, or Strong — the same kind of logic used in real security tools and login forms.

Strength Criteria

Check	Requirement
Length	At least 8 characters
Number	Contains at least one digit (0–9)
Uppercase	Contains at least one A–Z letter
Lowercase	Contains at least one a–z letter
Special char	Contains at least one of: ! @ # \$ % ^ & * ()

Code

```
password = input("Enter Password: ")

has_length = len(password) >= 8
has_number = any(ch.isdigit() for ch in password)
has_upper = any(ch.isupper() for ch in password)
has_lower = any(ch.islower() for ch in password)
has_special = any(ch in "!@#$$%^&*" for ch in password)

score = sum([has_length, has_number, has_upper, has_lower, has_special])

if score <= 2:
    print("Weak Password")
elif score <= 3:
    print("Moderate Password")
else:
    print("Strong Password")
```

Step-by-Step Explanation

- 1 `len(password) >= 8` returns True or False. That boolean value is stored directly in `has_length`.
- 2 `any(ch.isdigit() for ch in password)` loops through every character. `isdigit()` returns True for digit characters. `any()` returns True as soon as it finds one match and stops early — it does not waste time checking the rest.
- 3 The same pattern repeats for uppercase (`isupper()`), lowercase (`islower()`), and special characters (in `"!@#..."`). Each result is a True or False stored in its own variable.
- 4 `sum([...])` adds up the five boolean values. In Python, True counts as 1 and False counts as 0 — so the score is simply how many checks passed out of 5.

The final if / elif / else maps the score to a strength label: 0–2 = Weak, 3 = Moderate, 4–5 = Strong.

Sample Output

```
Enter Password: Cyber123
```

```
-- Checks -----
Length >= 8 chars      : PASS
Contains number        : PASS
Contains uppercase     : PASS
Contains lowercase     : PASS
Contains special char  : FAIL
-----
```

```
Result: Strong Password
```

Key Concepts

Concept	How it is used
any()	Returns True if at least one element in the sequence satisfies the condition
Generator expression	A compact loop written inline: <code>ch.isdigit() for ch in password</code>
str methods	<code>isdigit()</code> , <code>isupper()</code> , <code>islower()</code> classify individual characters
sum() on booleans	True = 1 and False = 0, so sum counts how many checks passed
if / elif / else	Maps the numeric score to a human-readable strength label